Настройка сервера

План занятия

1. Что требуется установить.
2. Настройка SSH.
3. Установка Docker.
4. Перенос БД.
5. Обновление кода приложения для работы с SQLite на сервере.
6. Создание образа и его запуск на сервере.

Что требуется установить

В прошлых занятиях мы уже настраивали SSH, устанавливали Docker и использовали концепцию CI/CD на практике. Нам предстоит повторить этот процесс для разработанного чат-бота, а также добавить в процесс юнит-тесты, базу данных и работу с переменными.

В процессе разработки собственного чат-бота, настройки окружения и процессов вы можете пользоваться этим уроком как руководством к действию, адаптируя процесс под свои условия и проект.

Для начала определим, что требуется установить или настроить на сервере:

- **SSH** — для удаленного подключения к серверу и его настройки. В целом, можно использовать способ авторизации по логину и паролю, но мы уже умеем настраивать более безопасный способ авторизации — по приватному и публичному ключу. Плюс этот вид авторизации пригодится нам при настройке CI/CD в нашем GitHub.

- **Docker** — скачивание, запуск и управление контейнерами.

- **SQLite** — для хранения данных в процессе работы чат-бота. Важный момент: SQLite не имеет сетевого интерфейса, база работает непосредственно из файла и взаимодействовать придется именно с ним напрямую. Поскольку при реализации CI/CD контейнер будет удаляться и создаваться из нового образа, потребуется вынести хранимую базу из контейнера, при этом обеспечив к ней

доступ из контейнера. Иными словами, база будет храниться на сервере вне контейнера, а из контейнера будет реализован доступ к этому файлу. Реализуется это с помощью инструментария Docker.

Важно. В данном материале представлены команды для использования в Ubuntu. Команды в Debian, CentOS и прочих операционных системах могут несколько отличаться.

Настройка SSH

Необходимо установить SSH-соединения с сервером, используя консоль на локальном компьютере. Процесс настройки SSH может отличаться от сервера к серверу, поэтому необходимо опираться на документацию каждого конкретного сервера индивидуально.

Рассмотрим, как примерно выглядит процедура настройки SSH.

Шаг 1. Создание SSH-ключей на локальном компьютере — например, с помощью команды в консоли:

```
ssh-keygen -t ed25519
```

После выполнения этой команды будет предложено ввести имя файла, в котором будет сохранен ваш ключ. По умолчанию это будет **id_rsa** для закрытого ключа и **id_rsa.pub** для открытого ключа.

Шаг 2. Открытый ключ (id_rsa.pub) необходимо скопировать на сервер в файл ~/.ssh/authorized_keys, чтобы разрешить подключение к этому серверу.

Шаг 3. После добавления открытого ключа на сервер необходимо подключиться к серверу с помощью сертификата, воспользовавшись командой:

```
ssh -i <key_path/key_file_name> <username>@<VM_public_IP_address>
```

Здесь вместо <key_path/key_file_name> нужно указать путь к файлу с приватным ключом, а вместо <username>@<VM_public_IP_address> указать имя пользователя и IP-адрес сервера.

Например:

```
ssh -i /Users/alexeymalyshko/.ssh/id_rsa_mipt  admin@10.192.18.21
```

Шаг 4. После ввода команды должно быть открыто SSH-соединение без ввода пароля от пользователя сервера. Перед установкой соединения будет запрошено подтверждение создания fingerprint — нужно подтвердить вводом слова **yes** в консоли в ответ на запрос.

Шаг 5. В случаях, когда подключение по SSH происходит без указания флага -i и указания пути, требуется указать ключ (в файле known_hosts), который будет использоваться для подключения к конкретному серверу. Для этого требуется ввести команду:

```
nano ~/.ssh/known_hosts
```

Затем в открывшемся файле нужно найти строку, которая начинается с IP-адреса нашего сервера или создать такую строку самостоятельно. В строку требуется добавить не только IP-адрес, но и публичный ключ, пару которого необходимо передать на сервер для аутентификации.

Строка должна иметь формат:

```
your_server_ip ssh-rsa AAAAB3NzaC1yc2E...
```

Вместо your_server_ip необходимо вставить IP-адрес нашего сервера, а после ssh-rsa должен идти открытый SSH-ключ, который был сгенерирован на первом этапе этой инструкции.

Для того чтобы вывести в консоль открытый ключ, который был создан, следует ввести команду:

```
cat ~/.ssh/id_rsa.pub
```

После внесения изменений в файл known_hosts требуется сохранить изменения с помощью нажатия на сочетание клавиш Ctrl + X.

Результат: настроенная авторизация по SSH на сервере.

Установка Docker

Чтобы установить Docker на сервер, нужно последовательно выполнить следующие команды в терминале:

```
sudo apt-get update

sudo apt-get install ca-certificates curl gnupg

sudo install -m 0755 -d /etc/apt/keyrings

curl -fsSL https://download.docker.com/linux/debian/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo \
  "deb [arch="$(dpkg --print-architecture)" signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/debian \
  "$(. /etc/os-release && echo "$VERSION_CODENAME")" stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt-get update

sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

sudo docker run hello-world
```

Приведенная выше инструкция взята из официальной инструкции по установке Docker для Debian.

Если установка прошла успешно, то в консоли появится информация об этом.

Результат: установленный Docker на сервере.

Перенос БД

Создание дампа БД

На локальном компьютере у нас уже создана БД (database.db), с которой работает наша локальная копия чат-бота.

Существует два пути при переносе БД на удаленный сервер:

- Создание новой пустой БД на сервере с необходимой структурой.
- Экспорт имеющейся локальной БД и импорт дампа на удаленный сервер.

В предыдущих уроках мы уже рассматривали процесс создания БД локально, и он практически не отличается от создания БД на удаленном сервере. Поэтому далее рассмотрим процесс экспорта и импорта.

При использовании SQLite не требуется делать экспорт и импорт БД в формате SQL. Достаточно просто перенести файл с базой данных на сервер. Ранее мы использовали название файла database.db. Этот файл и потребуется перенести на сервер.

Перенос БД на сервер

Перенести файл на сервер тоже можно несколькими способами. Разберем наиболее популярные:

- **SSH и SCP.** Подходит, если в вашем распоряжении есть доступ к серверу по SSH. Метод состоит в использовании утилиты SCP (Secure Copy) для копирования файла дампа с вашего локального компьютера на сервер.

  Пример команды для копирования файла на сервер:

  ```
  scp /path/to/exported_database.sql user@your_server_ip:/path/on/server/
  ```

  Здесь **user** — имя пользователя на сервере, **your_server_ip** — IP-адрес сервера, **/path/to/exported_database.sql** — путь к вашему файлу дампа, а **/path/on/server/** — путь на сервере, куда нужно скопировать файл.

- **FTP/SFTP.** Если сервер поддерживает протокол FTP или SFTP, вы можете использовать FTP/SFTP-клиент для передачи файла дампа на сервер. Примеры популярных FTP-клиентов: FileZilla, Cyberduck и CrossFTP.

- **Git, панели администрирования, облачные хранилища и другие способы.**

Рекомендация: используйте FTP/SFTP-клиенты для большей наглядности процесса переноса файла на сервер.

Внутри будущего контейнера мы разместим нашу базу данных в папке /root/db, однако не все так просто. Поскольку хранить базу внутри контейнера не получится по причинам, которые изложены выше, потребуется создать директорию с данной базой на сервере, за рамками контейнера.

Вернемся в корневую директорию сервера, введя команду:

```
cd
```

Создадим директорию, в которой будет храниться БД:

```
sudo mkdir app_data
```

Теперь с помощью файлового менеджера перенесем файл database.db в эту директорию. Но из-за того, что сам код приложения будет исполняться в контейнере, по умолчанию воспользоваться размещенными на сервере файлами не получится. Потребуется примонтировать папку, чтобы она стала доступна из контейнера, несмотря на то, что фактически она размещена на основном томе сервера.

Производится это в момент создания контейнера с помощью команды:

```
sudo docker run -d \
  -v <path_to_server_folder>:<path_to_containter_folder> \
  --name <container_name>  \
  <image_name> \
```

Где **<path_to_server_folder>** — путь до папки на стороне сервера, **<path_to_containter_folder>** — путь до папки в контейнере, **<container_name>** — название создаваемого контейнера, **<image_name>** — образ, из которого будет создаваться контейнер.

Пример команды:

```
sudo docker run -d -it \
```

```
-v /home/access/app_data:/app/app_data \
--name main_container \
malyshkoa2/chat-bot-mipt
```

Запомним эту команду и перейдем к модификации кода нашего приложения, чтобы после публикации кода на сервере он корректно исполнялся.

Результат: БД установлена на сервере и готова к монтированию в контейнер.

Обновление кода приложения для работы с SQLite на сервере

Нам потребуется слегка модифицировать все обращения к базе данных в основном исполняемом файле, а также файле с тестами. Эти модификации будут призваны скорректировать адрес, по которому будет располагаться файл с БД.

Изменим все фрагменты кода:

```
conn = sqlite3.connect('database.db')
```

на:

```
conn = sqlite3.connect('./app_data/database.db')
```

Теперь код приложения будет ожидать, что файл с базой данных будет размещаться в директории **app_data**, которая будет размещаться в корневой директории. Напомню, что в момент создания контейнера мы будем монтировать две директории с помощью фрагмента **-v /root/tmp:/app_data/**. Таким образом, ожидаемый кодом файл БД будет на месте.

Важно. Нужно заменить указанные фрагменты кода не только в основном исполняемом файле, но и в файле с юнит-тестами.

Перенеся код на сервер с помощью FTP/SFTP-клиента, следует сделать проверку. Основной файл с ботом должен запускаться, а тесты успешно проходиться.

Результат: обновленный код приложения, позволяющий подключаться к БД, расположенной в директории /app_data/.

Создание образа и его запуск на сервере

На данном этапе нами должны быть настроены SSH и Docker на сервере, а код приложения должен быть изменен таким образом, чтобы подключаться к БД, размещенной в директории ./app_data/database.db. Точка в начале пути означает его относительность. Иными словами, этот путь интерпретируется так: <папка с проектом>/app_data/database.db.

Теперь нам предстоит создать образ и отправить его в DockerHub, чтобы перед этапом настройки CI/CD через GitHub убедиться, что все необходимые компоненты работают отдельно от GitHub.

Вспомним из предыдущих занятий, как создается Docker-образ.

Шаг 1. Скачать клиентское приложение Docker для своей операционной системы и установить его.

Шаг 2. Создать аккаунт в DockerHub.

Шаг 3. Создать новый публичный репозиторий.

Шаг 4. Запустить установленное клиентское приложение Docker и авторизоваться.

Шаг 5. Добавить в директорию с проектом файл .dockerignore, в котором следует объявить файлы и папки, которые требуется игнорировать при создании образа:

```
.gitignore
/app_data
```

Шаг 6. Скачать готовый Docker-файл или создать его самостоятельно в папке приложения с названием Dockerfile, без расширения.

Шаг 7. Поместить в созданный файл следующий фрагмент текста:

```
# Установка Python из официального базового образа
FROM python:3.11

# Установка рабочей директории внутри будущего контейнера
```

```
WORKDIR /app

# Копирование всех файлов приложения в контейнер
COPY . /app
RUN pip install --no-cache-dir -r requirements.txt

# Экспорт порта, на котором будет работать приложение
EXPOSE 8000

# Запуск тестового Python-приложения
CMD ["python3", "main.py"]
```

Шаг 8. Создать Docker-образ, перейдя в папку с приложением, содержащим созданный ранее Dockerfile, с помощью терминала MacOS или Windows. А затем запустить там команду:

```
docker build --platform linux/amd64 -t <docker_hub_username>/<image_name>
<path_to_local_app_folder>
```

Где: **<docker_hub_username>** — имя пользователя в DockerHub, **<image_name>** — имя образа, **<path_to_local_app_folder>** — локальный путь до папки с приложением.

Например:

```
docker build --platform linux/amd64 -t malyshkoa2/chat-bot-mipt
/Users/alexeymalyshko/Documents/Self/Projects/chat-bot-mipt
```

Шаг 9. Запустить загрузку и запуск контейнера с помощью команды в терминале сервера:

```
docker run -d -it \
  -v /home/access/app_data:/app/app_data \
  --name main_container \
  malyshkoa2/chat-bot-mipt
```

В результате чат-бот будет запущен.

Чтобы подключиться к консоли контейнера, можно ввести:

```
sudo docker exec -it <container_id> /bin/bash
```

Можно узнать идентификатор контейнера, если ввести команду:

```
sudo docker ps
```

Рабочая папка внутри образа устанавливается благодаря Dockerfile — /app/. Затем при запуске контейнера с помощью -v /root/tmp/app_data:/app/app_data мы монтируем папку app_data в папку /app/. Таким образом в контейнере появляется доступ к файлу БД. А поскольку код нашего приложения обращается к файлу БД именно по этому адресу, все работает!

Результат: Docker-контейнер, запущенный на сервере. Чат-бот взаимодействует с БД, примонтированной из внешней директории на сервере.

Итоговый код проекта из занятия можно скачать по ссылке.

Заключение

В этом занятии мы разобрали процесс настройки SSH и Docker, а также научились монтировать папки основного хранилища сервера к контейнерам. Это позволило нам предоставить доступ к БД, которая не находится внутри контейнера.

В предстоящих занятиях мы настроим полный CI/CD с помощью GitHub, а также добавим юнит-тесты в пайплайн.